

|                            |                                              |
|----------------------------|----------------------------------------------|
| FEUP-L.EIC024-2025/2026-1S |                                              |
| Estado                     | Prova submetida                              |
| Início                     | quarta-feira, 14 de janeiro de 2026 às 15:19 |
| Data de submissão:         | quarta-feira, 14 de janeiro de 2026 às 16:48 |
| Tempo gasto                | 1 hora 28 minutos                            |
| Avaliação                  | 7,15 de um máximo de 20,00 (35,75%)          |

Informação

Consider a knowledge base for a bird-watching club called PFL (from the Portuguese *'Pássaros Fantásticos para Lunáticos'*).

PT

The *bird/4* predicate, ***bird(Name, Species, Gender, Colors)***, represents a specific bird watchable in the club, defined by its name, species, gender and the colors of its feathers (in decreasing order of body surface area). Assume all bird names are unique.

```
bird(robinho, robin, male, [red, brown, white]).
bird(robina, robin, female, [brown, red, white]).
bird(ferrugem, robin, male, [brown, gray]).

bird(arcoiris, parrot, male, [red, blue, green, yellow]).
bird(verdeja, parrot, female, [green, yellow]).

bird(minerva, owl, female, [brown, white]).
bird(noctis, owl, male, [gray, white]).
bird(sabia, owl, female, [beige, brown]).
```

In questions 1 to 5 you cannot use multiple solution predicates (findall, setof, and bagof) nor any SICStus library.

## Pergunta 1

Pontuou 1,000 de 1,000

Implement **male(?Name)**, which succeeds if the bird called **Name** is male. Example:

PT

```
| ?- male(B).  
B = robinho ? n  
B = ferrugem ?  
yes
```

```
male(Name) :-  
    bird(Name, _Species, Gender),  
    Gender = male.
```

```
male(Name):-  
    bird(Name, _, male, _).
```

Comentário:

Automatic ['O', 'O', 'O', 'O'] Max grade for this attempt is 1.0 out of 1.

## Pergunta 2

Pontuou 1,200 de 2,000

Implement ***has\_more\_color\_of(?Name, ?GreaterColor, ?LesserColor)***, which succeeds if bird ***Name*** has more of the color ***GreaterColor*** than of ***LesserColor*** (i.e. ***GreaterColor*** comes before ***LesserColor*** in the bird's color list).

PT

Constraint: In this exercise, you cannot define recursive predicates. Solutions that do not follow this constraint can obtain at most 25% of the question's maximum grade.

Suggestion: use *append/3*. Example:

```
| ?- has_more_color_of(arcoiris, red, green).
true ?
yes
| ?- has_more_color_of(robina, white, red).
no
```

```
has_more_color_of(Name, GreaterColor, LesserColor):-
    bird(Name, _Species, _Genus, _Colors),
    append([GreaterColor|_], _Colors, _),
```

```
has_more_color_of(N, C1, C2):-
    bird(N, _, _, Cs),
    append(_, [C1|T], Cs),
    append(_, [C2|_], T).
```

Comentário:

Automatic ['O', 'O', 'X', 'O', 'O', 'X', 'X', 'O', 'X', 'O', 'O', 'O', 'O', 'O', 'X', 'O', 'O', 'O', 'O', 'X', 'O', 'O', 'X', 'X', 'O', 'O', 'O', 'X', 'O', 'X', 'O', 'O', 'O', 'O', 'O', 'O', 'X'] Matched pattern ['O', 'O', 'NOE', 'O', 'O', 'NOE', 'NOE', 'O', 'NOE', 'O', 'O', 'O', 'O', 'O', 'NOE', 'O', 'O', 'O', 'O', 'NOE', 'O', 'O', 'NOE', 'NOE', 'O', 'O', 'O', 'NOE', 'O', 'NOE', 'O', 'O', 'O', 'O', 'O', 'O', 'NOE'], assigned 0.6 of max grade. Comment: Assumes GreaterColor is at the beginning of the colors list. Max grade for this attempt is 2.0 out of 2.

## Pergunta 3

Pontuou 1,600 de 2,000

Implement **most\_colorful(?Species, ?Name, ?NColors)**, which succeeds if **Name** is the name of the bird with the highest number of colors (returned in **NColors**) of species **Species**.

PT

In case of a tie, each bird with the maximum **NColors** should be returned via backtracking.

Suggestion: use the \+/1 (not) operator. Example:

```
| ?- most_colorful(robin, N, NC).
N = robinho,
NC = 3 ?
yes
```

```
most_colorful(Species, Name, NColors):-
    bird(Name, Species, _Gender,
          length(Colors, NColors),
    \+ (
        bird(Name, Species, _Gender,
              length(NewColors,
                    NewNColors > NColors),
    ).
```

```
most_colorful(Sp, N, NC):-
    bird(N, Sp, _, Cs),
    length(Cs, NC),
    \+((
        bird(_, Sp, _, Cs1),
        length(Cs1, NC1),
        NC1 > NC
    )).
```

Comentário:

[ 'O', 'O', 'O', 'O', 'X', 'O', 'X', 'X', 'O', 'O', 'O', 'O', 'X', 'O', 'O', 'O', 'X', 'O', 'O', 'O', 'O', 'O', 'X', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'X' ] Multiplied grade by 0.95 due to Sanitization: removed\_mode\_indicators. Max grade for this attempt is 1.9 out of 2.

- Forcing name and gender to be the same in second bird call

## Pergunta 4

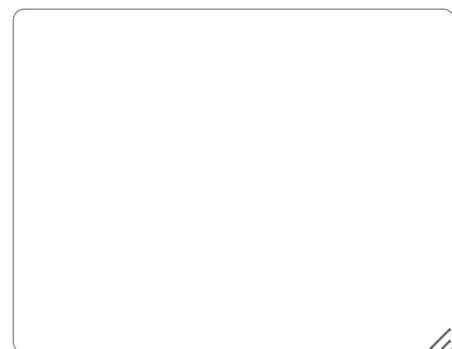
Pontuação 2,000

Implement **unique\_colors(+Species, -ListOfColors)**, which succeeds if **ListOfColors** is a list without duplicates of all colors found in at least one bird of species **Species**.

PT

Any order of the colors in **ListOfColors** will be accepted. Example:

```
| ?- unique_colors(owl, L).  
L = [beige,gray,white,brown] ?  
yes
```



```
unique_colors(Sp, L):-  
    unique_colors_aux(Sp, [], L).  
  
unique_colors_aux(Sp, Acc, Sol):-  
    bird(_, Sp, _, Cs),  
    member(C, Cs),  
    \+ member(C, Acc),  
    !,  
    unique_colors_aux(Sp, [C|Acc], Sol).  
unique_colors_aux(_, Acc, Acc).
```

## Pergunta 5

Pontuou 1,600 de 2,000

Implement ***is\_color\_permutation(?Name1, ?Name2)***, which succeeds if the colors of two distinct birds ***Name1*** and ***Name2*** are permutations of one another, i.e., the lists of colors of ***Name1*** and ***Name2*** have the same values, possibly in a distinct order. Example:

PT

```
| ?- is_color_permutation(robinho, robina).
yes ?
```

```
list_perm([],_).

list_perm([H|T], L) :-
    member(H,L),
    list_perm(T,L).

is_color_permutation(Name1, Name2) :-
    bird(Name1, _, _, Colors1),
    bird(Name2, _, _, Colors2),
    list_perm(Colors1,Colors2).
```

```
is_color_permutation(N1, N2) :-
    bird(N1, _, _, Cs1),
    bird(N2, _, _, Cs2),
    N1 \= N2,
    sort(Cs1, Sorted),
    sort(Cs2, Sorted).
```

## Comentário:

Automatic ['O', 'O', 'O', 'O', 'O', 'X', 'O', 'O', 'X', 'X', 'X', 'O', 'X', 'X', 'X', 'O', 'I', 'X', 'O', 'O', 'O'] Matched pattern ['O', 'O', 'O', 'O', 'O', 'NOE', 'O', 'O', 'NOE', 'NOE', 'NOE', 'O', 'NOE', 'NOE', 'NOE', 'O', 'NOE', 'NOE', 'O', 'O', 'O'], assigned 0.8 of max grade. Comment: Does not test if the two birds are distinct. Max grade for this attempt is 2.0 out of 2.

## Informação

In the following questions, you can use multiple solution predicates (findall, setof, and bagof) as well as the lists library.

PT

## Pergunta 6

Pontuação 2,000

Implement ***dif\_n\_colors(+Species, ?D)***, which succeeds if ***D*** is the difference between the maximum and minimum number of colors among the birds of species ***Species***. Example:

PT

```
| ?- dif_n_colors(robin, D).
D = 1 ?
yes
```

```
dif_n_colors(Sp, D):-
    findall(N, ( bird(_, Sp, _, Cs), length(Cs, N) ), L),
    sort(L, Sorted),
    last(Sorted, Max),
    Sorted = [Min|_],
    D is Max - Min.
```

## Pergunta 7

Pontuação 2,500

Implement ***most\_common\_color\_per\_species(?Species, ?Color)***, which succeeds if ***Color*** is the most common color among all birds of species ***Species***.

PT

In case of a tie, return each of the colors via backtracking. Example:

```
| ?- most_common_color_per_species(Sp, Color).
Sp = owl, Color = brown ? n
Sp = owl, Color = white ? n
Sp = parrot, Color = green ?
yes
```

```
most_common_color_per_species(Sp, Color):-
    bagof(C, (N,G,Cs)^( bird(N, Sp, G, Cs), member(C, Cs) ), Colors),
    setof(N-C, (Rest,All,LRest)^( member(C, Colors), delete(Colors, C, Rest), length(Colors, All), length(Rest, LRest), N is All-LRest ), Counts),
    last(Counts, MaxN-_),
    member(MaxN-Color, Counts).
```

## Pergunta 8

Pontuação 2,500

The bird-watchers want to find a viewing sequence from an initial bird to another bird so that when gazing at the birds through their binoculars they can manage to see a nice sequence of creatures.

PT

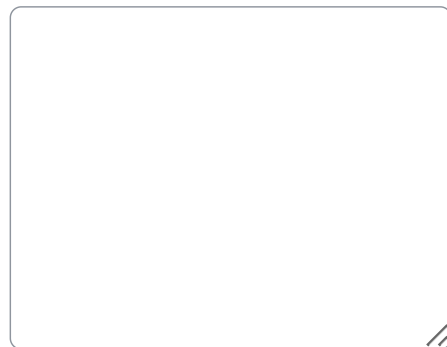
Implement **colorful\_routes(+Ni, +Nf, ?L)** which succeeds if **L** is a 'colorful route' between bird **Ni** and bird **Nf** (identified by their names).

A 'colorful route' is an ordered sequence of bird names starting at **Ni** and ending at **Nf**, where:

- a - No bird appears more than once in the route;
- b - At least 5 birds are visited, including **Ni** and **Nf**;
- c - Each pair of consecutive birds have two distinct main colors (i.e. the heads of the color list of two adjacent birds are different, so after visiting a bird that is mainly blue, the next bird cannot be mainly blue, but it can still have some blue feathers).

Suggestion: use depth-first search (dfs) to explore the possible routes. Example:

```
| ?- colorful_routes(robina, robinho, L).
L = [robina,arcoiris,ferrugem,verdeja,robinho] ?
yes
```



```
colorful_routes(Ni, Nf, L):-
    L = [_,_,_,_,_ | _],
    dfs([Ni], Nf, L).

dfs([Nf|_], Nf, [Nf]).
dfs([Na|Acc], Nf, [Na|Path]):-
    bird(Na,_,_,[Ca|_]),
    bird(Nb,_,_,[Cb|_]),
    Ca \= Cb,
    \+ member(Nb, Acc),
    dfs([Nb,Na|Acc], Nf, Path).
```



## Pergunta 9

Pontuou 0,500 de 0,500

Consider the following Prolog code snippet:

PT

```
predX:-
    read(X),
    atom(X),
    !,
    predX_aux(X).

predX_aux(X):-
    bird(N, _, X, _),
    write(N), nl,
    fail.
predX_aux(_).
```

- a. *predX/0* starts by asking the user to write one or more characters in the console, followed by '.' (full-stop). ☒
- b. *predX/0* starts by asking the user to write one character in the console, followed by ',' (comma). ☐
- c. *predX/0* starts by asking the user to write one or more characters in the console, followed by ';' (comma). ☐
- d. *predX/0* starts by asking the user to write one character in the console, followed by '.' (full-stop). ☐

A sua resposta está correta.

Resposta correta:

*predX/0* starts by asking the user to write one or more characters in the console, followed by '.' (full-stop).

## Pergunta 10

Pontuou -0,125 de 0,500

Still considering the Prolog code snippet from the previous question, the predicate *predX\_aux/1*:

PT

- a. always fails. ☐
- b. always succeeds. ☐
- c. succeeds if X is a valid bird species. ☐
- d. succeeds if there is a bird with the name read from the console. ☒

A sua resposta está incorreta.

Resposta correta:

always succeeds.

## Pergunta 11

Pontuação 0,500

Consider that one wants to define operators to represent a bird's cost, in order to write facts such as:

PT

`budgie costs 20 euros.`

What is the most accurate form to syntatic and semantically define the operators "costs" and "euros"?

- a. ☐ `:-op(550, xfy, costs).`  
☐ `:-op(540, xf, euros).`
- b. ☐ `:-op(550, xfx, costs).`  
☐ `:-op(570, xf, euros).`
- c. ☐ `:-op(550, fx, costs).`  
☐ `:-op(570, xf, euros).`
- d. ☐ `:-op(550, xfx, costs).`  
☐ `:-op(540, xf, euros).`

A sua resposta está incorreta.

Resposta correta:

`:-op(550, xfx, costs).`

`:-op(540, xf, euros).`

## Pergunta 12

Pontuou -0,125 de 0,500

Consider the following pairs of Prolog terms:

PT

- (1) *father(X, john)* and *daughter(mary, Y)*  
 (2) *likes(X, pizza)* and *likes(john, Y)*  
 (3) *f(X, X)* and *f(a, b)*  
 (4) *g(X, h(Y))* and *g(h(Z), h(a))*

Which one of the following options is correct?

- a. ☒ 2 and 3 are unifiable, but 1 and 4 aren't
- b. ☐ 1 and 2 are unifiable but 3 and 4 aren't
- c. ☐ none of these pairs of terms are unifiable
- d. ☐ 2 and 4 are unifiable but 1 and 3 aren't

A sua resposta está incorreta.

Resposta correta:

2 and 4 are unifiable but 1 and 3 aren't

## Pergunta 13

Pontuou 0,500 de 0,500

Consider the Prolog terms  $[X, a/T]$  and  $[b, Y, X]$ .

PT

Which of the following terms is a common instance of the two terms above?

- ☐ a.  $[b, a, a]$
- ☒ b.  $[b, a, b]$  ✓
- ☐ c. there is no common instance (i.e. the unification algorithm fails)
- ☐ d.  $[b, a, c]$

A sua resposta está correta.

Resposta correta:

$[b, a, b]$

## Pergunta 14

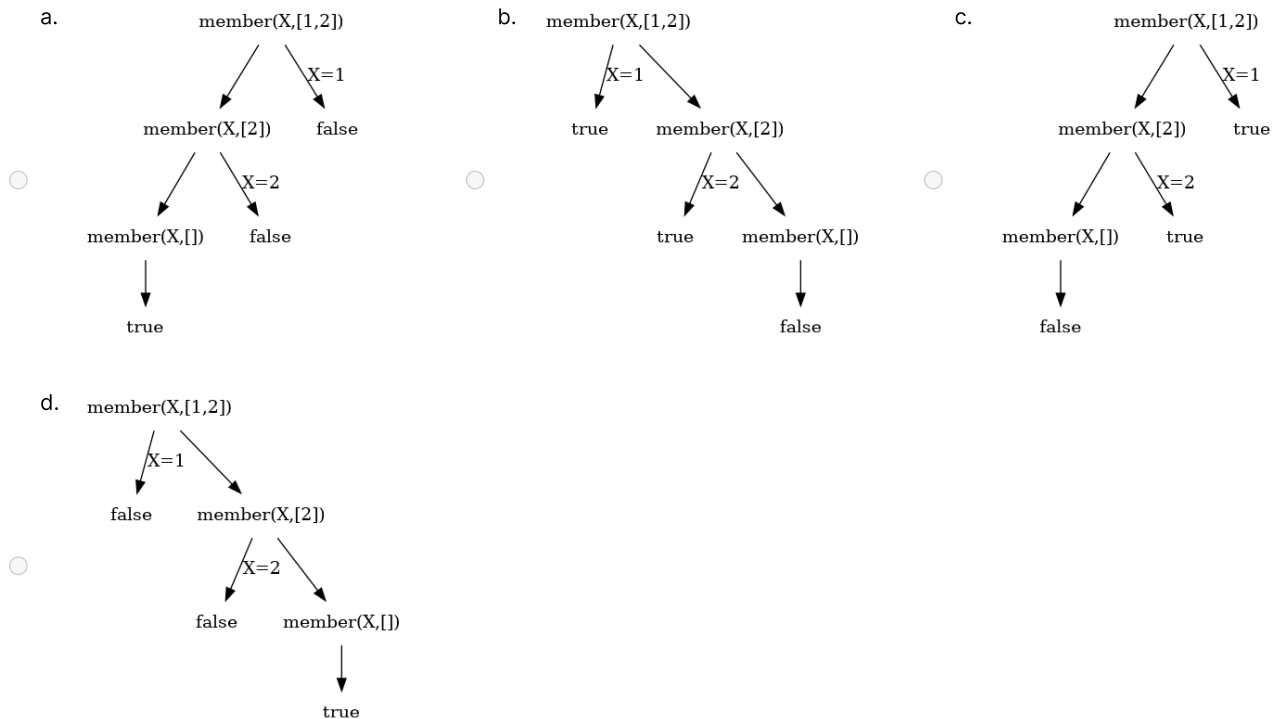
Pontuação 0,500

Consider the following Prolog definition for the *member/2* predicate:

```
member(X, [X|_]).
member(X, [_|T]) :- member(X,T).
```

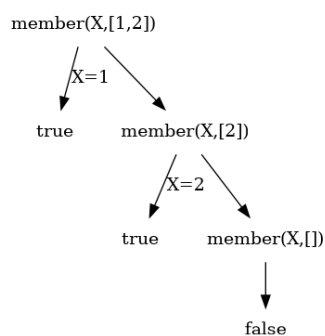
PT

Which of the following diagrams depicts the Selective Linear Definite (SLD) search tree for the goal  $?- \text{member}(X, [1,2])$ . ?



A sua resposta está incorreta.

Resposta correta:



## Pergunta 15

Pontuou 0,500 de 0,500

Consider the canonical definition of the *append/3* predicate.

Which of the following alternatives using *append/3* correctly defines the predicate *member/2*?

PT

- a. ☒ `member(X, Ys):- append(_ , [X|_] , Ys).` 
- b. ☐ `member(X, Ys):- append(_ , X , Zs), append(Zs, _ , Ys).`
- c. ☐ `member(X, Ys):- append([X] , _ , Ys).`
- d. ☐ `member(X, Ys):- append(_ , [X] , Ys).`

A sua resposta está correta.

Resposta correta:

`member(X, Ys):- append(_ , [X|_] , Ys).`

## Pergunta 16

Pontuou 0,500 de 0,500

Consider the following recursive definition of a *merge/3* predicate for merging two ordered lists of numbers into an ordered list of numbers with the combined values of both lists.

PT

```
merge([X|Xs], [Y|Ys], [X|R]) :-
    X<Y, !, merge(Xs, [Y|Ys], R).
merge([X|Xs], [Y|Ys], [X,Y|R]) :-
    X=Y, !, merge(Xs,Ys,R).
merge([X|Xs], [Y|Ys], [Y|R]) :-
    X>Y, !, merge([X|Xs], Ys, R).
merge(Xs, [], Xs).
merge([], Ys, Ys).
```

Which one of the following options is correct?

- a. ☐ All of the cuts in the definition above are red
- b. ☐ Whether the cuts are red or green will depend on the query to *merge/3*
- c. ☐ Some of the cuts in the definition above are green and some are red
- d. ☒ All cuts in the definition above are green 

A sua resposta está correta.

Resposta correta:

All cuts in the definition above are green

◀ Mini-Test 2 (2026/01/14) (Part 1 - Practical Assignment)

MT2, TP2 and final grades ▶